

Programming in Modern C++: Assignment Week 6

Total Marks :

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology
Kharagpur – 721302
partha.p.das@gmail.com

August 22, 2025

Question 1

Consider the code segment given below.

[MSQ, Marks 2]

```
#include <iostream>
using namespace std;

int main(){
    int *p;
    int x = 10;
    int y;
    p = &x;          // LINE-1
    y = *p;          // LINE-2
    *p = 20;          // LINE-3
    &x = p;           // LINE-4
    return 0;
}
```

Which line/s will give error?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: d)

Explanation:

Lines LINE-1, LINE-2, and LINE-3 are valid pointer operations: assigning address, dereferencing, and modifying the value. However, the expression `&x = p;` is invalid because an **Lvalue** reference (address of a variable) cannot be assigned. Hence only LINE-4 gives compilation error.

Question 2

Consider the code segment given below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class Base{
    public:
        void show(){ cout << "Base"; }
};

class Derived : public Base{
    public:
        void print(){ cout << "Derived"; }
};

int main(){
    Derived d;
    Base *b = &d;
    b->show();      //LINE-1
    b->print();     //LINE-2
    return 0;
}
```

What will be the output/error?

- a) BaseDerived
- b) Base
- c) Compilation error at LINE-1
- d) Compilation error at LINE-2: class Base has no member named 'print'

Answer: d)

Explanation:

Pointer b is of type Base*, so it can only access members of Base. Hence, b->show() is valid, but b->print() is invalid as Base does not have print(). Thus, compilation error occurs at LINE-2.

Question 3

Consider the following code segment.

[MSQ, Marks 2]

```
#include <iostream>
using namespace std;

class A{ };
class B{ };

int main(){
    A a;
    B b;
    A *p1;
    B *p2;
    p1 = (A*)&b;    //LINE-1
    p2 = (B*)&a;    //LINE-2
    a = b;          //LINE-3
    b = (B)a;       //LINE-4
    return 0;
}
```

Which line/s will give error?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: c), d)

Explanation:

Explicit casting between unrelated class pointers (A* and B*) is syntactically allowed (LINE-1 and LINE-2). However, assignment between unrelated class objects (a = b or (B)a) is not allowed and gives compilation errors at LINE-3 and LINE-4.

Question 4

Consider the code segment given below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class Parent{
    public:
        void display(){ cout << "Parent"; }
};

class Child : public Parent{
    public:
        ----- //LINE-1
        void display(int){ cout << "Child"; }
};

int main(){
    Child c;
    c.display();
    return 0;
}
```

Fill in the blank at LINE-1 such that the program runs successfully and prints Parent

- a) Parent::display();
- b) using Parent::display;
- c) Parent.display();
- d) void display();

Answer: b)

Explanation:

The display() function in Parent is hidden by the overload display(int) in Child. To make the base version visible, we use using Parent::display;. Thus, the call c.display() resolves to Parent::display().

Question 5

Consider the code segment given below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class Alpha{
public:
    Alpha(){ cout << "A "; }
    virtual ~Alpha(){ cout << "~A "; }
};

class Beta : public Alpha{
public:
    Beta(){ cout << "B "; }
    ~Beta(){ cout << "~B "; }
};

void func(Alpha a){ cout << "F "; }

int main(){
    Beta b;
    func(b);
    return 0;
}
```

What will be the output?

- a) A B A F ~A ~B ~A
- b) A B F ~A ~B ~A
- c) A B F ~B ~A
- d) A B F ~B

Answer: b)

Explanation:

Creating object **b** calls **Alpha()** first and then **Beta()** (prints A B). Calling **func** prints F. At the end of the function **func**, the local object **a** goes out of scope and is destroyed, which invokes the destructor of class A. At the end of the scope of **main**, the object **b** also needs to be destroyed, which invokes the destructors in the following order: $\sim B \rightarrow \sim A$. Therefore, the output is: A B F ~A ~B ~A.

Question 6

Consider the code segment below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class Abstract{
    public:
        -----;    //LINE-1
};

class Concrete : public Abstract{
    public:
        void show(){ cout << "Concrete"; }
};

int main(){
    Abstract *ptr = new Concrete;
    ptr->show();
    return 0;
}
```

Fill in the blank at LINE-1 such that the program generates output as
Concrete

- a) void show() = 0
- b) virtual void show()
- c) virtual void show() = 0
- d) void show()

Answer: c)

Explanation:

To allow polymorphism, `show()` must be declared virtual. Also, since `Abstract` is intended to be an abstract base class (pure virtual), the correct declaration is:
`virtual void show() = 0;`

Question 7

Consider the code segment below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class X{
    public:
        void fun(){ cout << "X "; }
};

class Y : public X{
    public:
        virtual void fun(){ cout << "Y "; }
};

class Z : public Y{
    public:
        void fun(){ cout << "Z "; }
};

int main(){
    X *p1 = new Z;
    Y *p2 = new Z;
    p1->fun();
    p2->fun();
    return 0;
}
```

What will be the output?

- a) Z Z
- b) X Y
- c) X Z
- d) Z Y

Answer: c)

Explanation:

Pointer p1 is of type X*. Since X::fun() is non-virtual, static binding occurs, so it calls X::fun(). Pointer p2 is of type Y*, and fun() is virtual in Y, so dynamic binding occurs, invoking Z::fun(). Hence output is X Z.

Question 8

Consider the code segment below.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class P{
    public:
        virtual void f(){ cout << "P::f"; }
};

class Q : public P{
    public:
        void f(){ cout << "Q::f"; }
};

class R : public Q{
    public:
        void f(){ cout << "R::f"; }
};

int main(){
    R *obj = new R;
    obj->Q::f();
    return 0;
}
```

What will be the output/error?

- a) P::f
- b) Q::f
- c) R::f
- d) Compilation error

Answer: b)

Explanation:

The statement `obj->Q::f()` explicitly calls the version of `f()` defined in class `Q`, bypassing dynamic dispatch. Thus, it prints `Q::f`.

Question 9

Consider the following code segment.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

class Base{
public:
    virtual void func1(){ cout << "Base::func1 "; }
    virtual void func2() = 0;
    void func3(){ cout << "Base::func3 "; }
};

class Derived : public Base{
public:
    void func1(){ cout << "Derived::func1 "; }
    void func2(){ cout << "Derived::func2 "; }
    void func3(){ cout << "Derived::func3 "; }
};

int main(){
    int n;
    cin >> n; // input for clarity but not affecting output
    Derived obj;
    Base *bptr = &obj;
    Base &bref = obj;

    bptr->func1();    //LINE-1
    bptr->func2();    //LINE-2
    bptr->func3();    //LINE-3

    bref.func1();    //LINE-4
    bref.func2();    //LINE-5
    bref.func3();    //LINE-6
}
```

Identify which function calls are resolved dynamically and which are resolved statically.

- a) Dynamic calls: Lines 1,2,4,5 Static Calls: Lines 3,6
- b) Dynamic calls: Lines 1,2,3 Static Calls: Lines 4,5,6
- c) Dynamic calls: Lines 1,3,4,6 Static Calls: Lines 2, 5
- d) None of the above

Answer: a)

Explanation:

The functions marked **virtual** (**func1()** and **func2()**) are dynamically bound (lines 1,2,4,5). The non-virtual **func3()** is statically bound (lines 3 and 6), regardless of using a pointer or reference to the base class.

Programming Questions

Question 1

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with appropriate function declaration for **Volume** function
- at LINE-2 with appropriate function declaration for **Print** function

such that it will satisfy the given test cases.

Marks: 3

```
#include <iostream>
#define PI 3.14
using namespace std;

class Shape{
protected:
    double dim;
public:
    Shape(double d) : dim(d) {}
    ----- //LINE-1
    ----- //LINE-2
};

double Shape::Volume(){ return dim*dim*dim; }
void Shape::Print(){ cout << Volume() << " "; }

class Sphere : public Shape{
public:
    Sphere(double r) : Shape(r) {}
    double Volume(){ return (4.0/3)*PI*dim*dim*dim; }
};

int main(){
    double r;
    cin >> r;
    Shape s(r);
    Sphere sp(r);
    Shape *arr[2] = {&s,&sp};
    for(int i=0;i<2;i++)
        arr[i]->Print();
    return 0;
}
```

Public 1

Input: 2

Output: 8 33.4933

Public 2

Input: 1

Output: 1 4.18667

Private

Input: 3

Output: 27 113.04

Answer:

LINE-1: `virtual double Volume();`

LINE-2: `virtual void Print();`

Explanation: Functions requiring polymorphic behavior must be declared virtual. `Volume()` is overridden in `Sphere`. `Print()` may also be virtual for flexibility.

Question 2

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with appropriate keyword,
- at LINE-2 with appropriate statement so that `Add()` can access private members of the class,
- at LINE-3 with appropriate statement

such that it will satisfy the given test cases.

Marks: 3

```
#include <iostream>
using namespace std;

class Alpha{
    int x;
    public:
        Alpha(int a) : x(a) {}
        ----- void display();                //LINE-1
        -----;                                //LINE-2
};

class Beta : public Alpha{
    int y;
    public:
        Beta(int a, int b) : Alpha(a), y(b) {}
        void display(){
            -----;                            //LINE-3
            cout << y << " ";
        }
};

void Alpha::display(){ cout << x << " "; }

void Add(Alpha &a1, Alpha &a2){
    a1.x = a1.x + a2.x;
}

int main(){
    int m, n;
    cin >> m >> n;
    Alpha *p1 = new Beta(m,n);
    Alpha *p2 = new Alpha(n);
    Add(*p1, *p2);
    p1->display();
    return 0;
}
```

Public 1

Input: 2 4

Output: 6 4

Public 2

Input: 3 5

Output: 8 5

Private

Input: 7 2

Output: 9 2

Answer:

LINE-1: `virtual`

LINE-2: `friend void Add(Alpha&, Alpha&)`

LINE-3: `Alpha::display()`

Explanation:

The `display()` function in `Alpha` should be declared as `virtual` so that `Beta`'s version gets called through a base class pointer. To allow `Add()` to access private member `x` of `Alpha`, it should be declared as a friend. Inside `Beta`'s `display()`, the base class version is explicitly invoked by `Alpha::display()` to print `x` before printing `y`.

Question 3

Consider the following program. Fill in the blanks as per the instructions given below:

- at LINE-1 with the correct declaration for the base class destructor,
- at LINE-2 with the correct declaration for the derived class destructor

such that it will satisfy the given test cases.

Marks: 3

```
#include <iostream>
using namespace std;

class Base{
protected:
    int val;
public:
    Base(int v);
    _____; //LINE-1
    int getVal() { return val; }
};

class Derived : public Base{
public:
    Derived(int v);
    _____; //LINE-2
};

Base::Base(int v) : val(v) { cout << val << " "; }
Base::~Base(){ cout << val*2 << " "; }
Derived::Derived(int v) : Base(v) { cout << val*3 << " "; }
Derived::~Derived(){ cout << getVal() << " "; }

int main(){
    int n;
    cin >> n; // Input value
    Derived *ptr = new Derived(n);
    Base *bptr = ptr;
    delete bptr;
    return 0;
}
```

Public 1

Input: 5

Output: 5 15 5 10

Public 2

Input: 2

Output: 2 6 2 4

Private

Input: 10

Output: 10 30 10 20

Answer:

LINE-1: `virtual ~Base()`

LINE-2: `~Derived()`

Explanation:

Since we are deleting the object using a base-class pointer, the base destructor must be **virtual** to ensure both destructors execute in the correct order. The derived class destructor declaration itself may remain non-virtual, but can optionally be marked virtual for clarity.